

Production-Grade RAG: A Step-by-Step Guided Walkthrough

1. Introduction: The "No-Framework" Philosophy

In the current AI landscape, it is tempting to reach for high-level abstractions like LangChain or LlamaIndex to "speed up" development. However, as a Senior AI Platform Architect, I have found that these frameworks often become liabilities in production. They introduce "magic" logic that is difficult to debug, brittle dependency axes, and opaque retrieval decisions. This walkthrough demonstrates a **direct-SDK approach**. By building directly against providers like OpenAI, Anthropic, and ChromaDB, we gain absolute control over the pipeline. Our objective is to build a system that is transparent, observable, and hardened for real-world deployment.

Why We Avoid High-Level Frameworks

- **Explicit Retrieval Decisions:** Every logic gate, from Reciprocal Rank Fusion (RRF) to context-budget trimming, is a plain Python function. You own the math, not the framework.
- **Minimal Dependency Friction:** We eliminate the "version-mismatch" nightmare where updating a framework breaks your vector store client or provider SDK.
- **Zero-Abstraction Debuggability:** When a query fails, your stack trace points exactly to `src/retrieval/hybrid.py`, not through twelve layers of internal "Chain" or "Agent" logic.
- **Interview-Defensible Design:** You aren't just "using an API"; you are orchestrating a platform. Every design choice—from 0o600 file permissions to SHA-256 prompt versioning—is intentional and visible.

High-Level Architectural Flow

The system processes data linearly:

1. **Ingestion:** Raw docs are loaded, recursively chunked, and indexed in parallel (Dense Vector + Sparse BM25).
2. **Query & Routing:** The system detects query language to route to the correct linguistic shard (en/de/es).
3. **Hybrid Retrieval:** We execute a two-pronged search, fused via RRF.
4. **Reranking:** A Cross-Encoder precisely re-orders candidates to minimize "lost-in-the-middle" issues.
5. **Generation:** The LLM produces a response with structured citations, governed by strict context-budget logic.

2. Phase 1: Building the Core RAG Baseline

Ingestion: Recursive Semantic Chunking

Production RAG begins with high-quality ingestion. We avoid simple fixed-length splitting, which often severs context in the middle of a sentence.

- **Implementation:** Use `loader.py` for document parsing and `chunker.py` for splitting.

- **Architectural Rationale:** We use a recursive strategy that attempts to split on paragraphs first, then sentences, and finally words. This ensures that chunks remain semantically coherent units.
- **Baseline Configuration:**
 - RAG_CHUNK_SIZE: 800 characters (the "goldilocks" zone for most technical docs).
 - RAG_CHUNK_OVERLAP: 150 characters (to ensure context isn't lost at the boundaries).

Storage: Dense Vector Store with ChromaDB

For our semantic search baseline, we utilize ChromaDB via `src/retrieval/vector_store.py`.

- **Logic:** Text chunks are passed to the embedding model, and the resulting vectors are stored in a collection specified by `RAG_CHROMA_PATH`.
- **Architect's Note:** We prioritize persistence. The vector store is configured to save to disk immediately, ensuring that the index survives container restarts—a mandatory requirement for production reliability.

Generation: Dual-Provider Backend & Sentence-Aware Trimming

The generation layer in `src/generation/generator.py` is designed for provider-agnostic flexibility.

LLM Provider Configuration Toggle between the industry leaders using the `RAG_LLM_PROVIDER` environment variable:

Variable	Value	Description
<code>RAG_LLM_PROVIDER</code>	<code>openai</code>	Defaults to gpt-4o-mini for cost-effective reasoning.
<code>RAG_LLM_PROVIDER</code>	<code>anthropic</code>	Uses Claude models for superior long-context handling.

Context Budget Management To prevent mid-sentence truncation—which confuses LLMs—our system implements "clean-cut" logic via `max_context_chars`. Before the context is passed to the LLM, the system trims retrieved chunks to fit the budget, but only at sentence boundaries. This preserves the logical flow of the evidence provided to the model.

3. Phase 2: Advanced Retrieval and Re-ranking

Hybrid Search and RRF Fusion

Semantic search often struggles with specific technical identifiers or acronyms. We solve this by implementing a hybrid retriever in `src/retrieval/hybrid.py`.

1. **BM25 Keyword Search:** We index the corpus using the `rank-bm25` library. To prevent memory exhaustion in production, we implement a **500 MB safety guard** on the index loader.
2. **Reciprocal Rank Fusion (RRF):** This algorithm merges the ranked lists from both vector and keyword searches.
3. **Balancing Search:** Use `RAG_HYBRID_ALPHA` (default 0.6) to tune the system. 1.0 is pure vector; 0.0 is pure keyword. **Security Note:** The BM25 index is serialized using **JSON**, not Python's pickle. In production, pickle is a massive security vulnerability (arbitrary code execution). We explicitly avoid it.

Cross-Encoder Re-ranking

Standard vector search is "bi-encoder" based—fast but less precise. We add a "Cross-Encoder" step in `src/retrieval/reranker.py` to act as a precision filter.

1. **The Workflow:**
2. **Fetch:** Retrieve the top 20 candidates (`RAG_TOP_K_RETRIEVAL`) using hybrid search.
3. **Score:** Use the cross-encoder/`ms-marco-MiniLM-L-6-v2` model to score the query against each chunk simultaneously.
4. **Sort & Select:** Re-order the chunks based on the new scores and pass only the top 5 (`RAG_TOP_K_FINAL`) to the LLM.

Multilingual Routing and Query Caching

To support global deployments, the system uses `langdetect` to shard documents into per-language ChromaDB collections (`en/de/es`).

- **i18n Prompts:** We don't just route the data; we use `gettext` helpers to switch the system prompts to the appropriate language, ensuring the LLM's "persona" remains consistent across regions.
- **LRU Query Cache:** To minimize latency and costs, an in-process LRU cache (`RAG_EMBEDDING_QUERY_CACHE_SIZE`, default 256) stores embeddings for common queries. This is critical for high-traffic environments where users often ask similar questions.

4. Phase 3: Evaluation, Observability, and Governance

LLM-as-Judge (RAGAS-Style Metrics)

We cannot improve what we cannot measure. Our evaluation suite in `src/evaluation/` uses the LLM-as-judge pattern to score quality dimensions.

Dimension	Measurement	Scoring Method
Faithfulness	Does the answer align with the context?	LLM-judged (0-1)
Answer Relevance	Does the answer address the user's intent?	LLM-judged (0-1)
Context Precision	Are relevant chunks ranked at the top?	Deterministic (Avg Precision @ k)
Context Recall	Does context cover the reference answer?	Deterministic (Attributable fraction)

OpenTelemetry and Prometheus

Monitoring is handled by the `MetricsCollector`, exposing a `/metrics` endpoint for Prometheus scraping.

- **Captured Metrics:** We track query latency histograms, query counts, error rates per pipeline step, and **USD cost tracking** based on token consumption.
- **Architect's Note:** Observability should never break the user experience. We implement a **Circuit Breaker** (`MONITOR_CIRCUIT_BREAKER_THRESHOLD`) so that if the telemetry backend fails, the user's query still completes successfully.

Prompt Registry and Versioning

In `src/monitoring/prompts.py`, we treat prompts as code. We use a **SHA-256 hash-based change detection system**. If a developer modifies a prompt text in the code, the system detects the hash mismatch, allowing for audited versioning across deployments.

5. Phase 4: Production API, Security, and Deployment

FastAPI and SSE Streaming

The API layer provides both synchronous and real-time streaming capabilities.

Method	Endpoint	Description
GET	/healthz	Liveness probe; open access.
GET	/readyz	Readiness probe; warms up models.
GET	/metrics	Prometheus endpoint for operational metrics.
POST	/query	Standard synchronous answer + citations.
POST	/query/stream	SSE Streaming ; delivers tokens in real-time.

Asynchronous Ingestion Job Management

For production corpora, ingestion can take minutes. To prevent proxy timeouts, we use a background job pattern:

1. POST `/ingest/async` returns a `job_id` immediately (202 Accepted).
2. The client polls GET `/ingest/jobs/{job_id}` for status. This keeps the API responsive even while indexing thousands of documents.

Security Hardening: Defense-in-Depth

- **Constant-Time Auth:** We use `secrets.compare_digest` for API key validation to prevent timing attacks.
- **ReDoS Prevention:** For pricing pattern matching, we use `re.escape()` before regex compilation to prevent Regular Expression Denial of Service.
- **File Permissions:** All sensitive data (BM25 indices, metrics summaries) are written with 0o600 permissions (owner read/write only).
- **Path Guards:** The `/ingest` endpoint validates that all source paths reside strictly within the approved `RAG_DATA_DIR`.

Deployment and Testing Strategy

- **Docker Strategy:** We use a multi-stage build (`deps -> model-cache -> runtime`). The models are baked into the image, eliminating "cold start" downloads. Crucially, the container runs as a **non-root user** for security.
- **Testing:** The system is backed by a **147-test suite** using `pytest`. This covers every logic gate from RRF math to the SSE streaming buffers, ensuring that "No-Framework" doesn't mean "No-Quality."

6. Conclusion: Scaling and Evolution

The architecture described here is designed for horizontal scale. Because the FastAPI service is stateless, you can spin up multiple replicas behind a load balancer immediately.

Future Roadmap:

- **Corpora Scaling:** As your data grows, transition from embedded ChromaDB to a managed cluster. Our per-language collection design already provides a natural sharding boundary.
- **Result Caching:** Implementing a cache for the final LLM response can further reduce costs for frequently asked questions. By choosing the "direct approach," you have built a system where every retrieval decision is explicit, every security measure is visible, and every latency spike is traceable. This is how production AI is built.